ESCUELA DE POSGRADO UNIVERSIDAD CONTINENTAL



MAESTRÍA EN INGENIERÍA DE SOFTWARE

MODALIDAD A DISTANCIA

MATERIA:

Desarrollo de Aplicaciones en la Nube

PRODUCTO ACADÉMICO COLABORATIVO:

Diseño de la arquitectura nativa en la nube de una plataforma de telemedicina para zonas rurales en Microsoft Azure: servicios gestionados, escalabilidad, seguridad y optimización de costos

AUTORES:

Jossip Jair Escalaya Juarez
Ivan Arturo Bolanos Victoria
Alejandro Andres Fernandez Alejandro
Carlos Eduardo Puma Mendoza

DOCENTE:

Marlon Luis Leandro Ramirez

2026 - PERÚ

Índice

1. Introducción....3
2. Desarrollo....3
2.1. Modelo de servicio seleccionado....3
2.2. Arquitectura de la aplicación....4
2.3. Servicios gestionados integrados....5
2.4. Estrategia de escalabilidad y automatización....6
2.5. Seguridad y cumplimiento....7
2.6. Optimización de costos....8
2.7. Prototipo implementado y verificación....9
2.8. Gestión ágil del desarrollo....11
3. Conclusiones....11
4. Referencias....12
Anexo A. Gestión ágil del desarrollo en Azure Boards....15
Anexo B. Issues y Tasks por sprint....26
Anexo C. Criterios de aceptación en Gherkin....29

1. Introducción

En el Perú, buena parte de la población rural vive a horas de distancia de un establecimiento de salud con médico especialista. La telesalud, reconocida por la Ley N.° 30421, Ley Marco de Telesalud, es una estrategia para reducir esa brecha; su uso creció de forma notable desde la pandemia de COVID-19 (Curioso & Galán-Rodas, 2020). El presente trabajo diseña la arquitectura de una plataforma de telemedicina para una organización sin fines de lucro que necesita ofrecer consultas virtuales, un historial clínico digitalizado y notificaciones de citas a pacientes de zonas rurales, con un presupuesto limitado y una demanda que cambia de forma brusca durante las campañas de salud.

La computación en la nube resulta adecuada para este escenario por tres razones. Primero, elimina la inversión inicial en servidores y la traslada a un gasto operativo que se paga según el uso (Mell & Grance, 2011). Segundo, ofrece elasticidad: la capacidad crece cuando una campaña atrae a miles de pacientes y se reduce cuando la demanda baja. Tercero, los proveedores ofrecen servicios gestionados con controles de seguridad, cifrado y certificaciones de cumplimiento que una ONG difícilmente podría construir por cuenta propia. Se eligió **Microsoft Azure** como proveedor y la propuesta sigue los cinco pilares del Azure Well-Architected Framework: confiabilidad, seguridad, optimización de costos, excelencia operativa y eficiencia del rendimiento (Microsoft, s.f.-b).

2. Desarrollo

2.1. Modelo de servicio seleccionado

Se adopta un modelo **PaaS con enfoque serverless**: la organización desarrolla y despliega su propio código (API, lógica de citas e historial), pero delega en Azure la administración de servidores, sistemas operativos, parches, balanceo y escalado. La Tabla 1 resume la comparación que sustenta la decisión.

Tabla 1

Comparación de modelos de servicio para el caso de telemedicina rural

Criterio	IaaS (máquinas virtuales)	PaaS / serverless (seleccionado)	SaaS (plataforma comercial)
Control y personalización	Total, incluido el sistema operativo	Alto sobre la aplicación y los datos	Bajo; limitado a la configuración del proveedor
Esfuerzo operativo	Alto: parches, respaldos y escalado manual	Bajo: el proveedor opera la plataforma	Mínimo
Escalabilidad	Requiere configurar grupos de escalado	Automática y por evento; escala a cero	Depende del plan contratado
Modelo de costo	Pago por hora aunque no haya uso	Pago por ejecución, solicitud o GB	Licencia por usuario o médico

Criterio	IaaS (máquinas virtuales)	PaaS / serverless (seleccionado)	SaaS (plataforma comercial)
Adecuación al caso	Costosa para cargas intermitentes	Ideal para demanda variable y equipo técnico pequeño	Poca flexibilidad para SMS, offline y normativa local

Nota. Elaboración propia a partir de Mell y Grance (2011) y Microsoft (s.f.-b).

Justificación técnica. La ONG no cuenta con un equipo grande de infraestructura; con PaaS sus pocos desarrolladores se concentran en las funciones clínicas y no en administrar servidores. Los servicios elegidos incluyen alta disponibilidad, cifrado y copias de seguridad sin configuración adicional. **Justificación económica.** La carga de una plataforma rural es intermitente: hay picos durante las campañas y valles en temporadas normales o de noche. Con IaaS se pagaría la capacidad del pico durante todo el mes; con serverless se paga solo lo consumido, y varios servicios incluyen cuotas mensuales gratuitas, como el millón de ejecuciones de Azure Functions en el plan Consumption o los primeros 50 000 usuarios activos mensuales de Microsoft Entra External ID (Microsoft, s.f.-a; Microsoft, s.f.-d). Se descarta SaaS porque una solución comercial cerrada no permitiría integrar SMS para teléfonos básicos, el funcionamiento sin conexión ni la interoperabilidad con el Registro Nacional de Historias Clínicas Electrónicas (RENHICE).

2.2. Arquitectura de la aplicación

La Figura 1 presenta la arquitectura de referencia. La solución se despliega en la región **Chile Central**, la más cercana al Perú, con tres zonas de disponibilidad, y guarda copias de respaldo en **Brazil South**. Sus componentes se organizan en cuatro capas y un plano transversal de seguridad y operación.

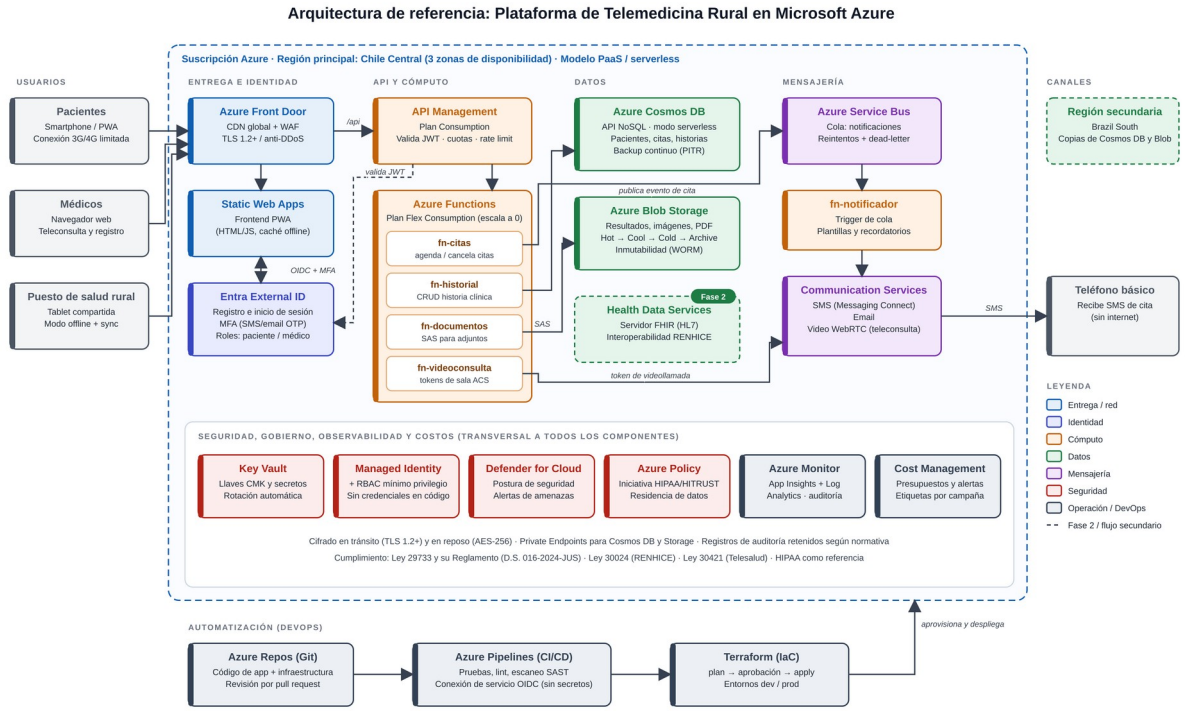
- **Capa de entrega e identidad.** Azure Front Door cachea el frontend, termina TLS y aplica reglas de firewall (WAF). El frontend es una aplicación web progresiva (PWA) en Azure Static Web Apps: se instala en el teléfono y funciona sin conexión en puestos de salud rurales. La autenticación se delega en Microsoft Entra External ID, que reemplazó a Azure AD B2C para nuevos clientes desde mayo de 2025 (Microsoft, s.f.-d).
- **Capa de API y cómputo.** Azure API Management (plan Consumption) publica una API REST única, valida el token JWT emitido por Entra y aplica cuotas por usuario. La lógica de negocio se implementa como microfunciones en Azure Functions (plan Flex Consumption): *fn-citas*, *fn-historial*, *fn-documentos* y *fn-videoconsulta*. Una fachada FHIR R4 expone a socios externos (hospital de referencia, DIRESA) los datos básicos del paciente y el estado de sus citas, sin contenido clínico (HL7 International, 2019).
- **Capa de datos.** Azure Cosmos DB (API NoSQL, modo serverless) almacena pacientes, citas e historias clínicas como documentos JSON particionados por

paciente. Los adjuntos (resultados de laboratorio, imágenes, PDF) se guardan en Azure Blob Storage y se acceden con URL firmadas (SAS) de corta duración. En la fase 2 se incorporará Azure Health Data Services con un servidor FHIR (HL7) para interoperar con el RENHICE.

- **Capa de mensajería.** Cada cita genera un evento en Azure Service Bus; una función desacoplada (*fn-notificador*) consume la cola y envía confirmaciones y recordatorios mediante Azure Communication Services (ACS), que además proporciona la videollamada WebRTC de la teleconsulta.

Figura 1

Arquitectura de referencia de la plataforma de telemedicina rural en Microsoft Azure



Nota. Elaboración propia. Las líneas discontinuas indican componentes de la fase 2 o flujos secundarios.

2.3. Servicios gestionados integrados

La propuesta integra más de los tres servicios gestionados requeridos. La Tabla 2 detalla los seis principales, su función y la razón de su elección.

Tabla 2

Servicios gestionados principales y justificación de su elección

Servicio	Función en la plataforma	Justificación de la elección
Microsoft Entra External ID	Registro, inicio de sesión, MFA y roles (paciente, médico, administrador)	Gestión de identidades para clientes (CIAM) con estándares OpenID Connect y OAuth 2.0; gratuito hasta 50 000 usuarios activos al mes (Microsoft, s.f.-d).

Servicio	Función en la plataforma	Justificación de la elección
Azure Functions (Flex Consumption)	Lógica de negocio de citas, historial, documentos y notificaciones	Escala por eventos y baja a cero instancias cuando no hay uso; es el plan serverless recomendado por Microsoft e incluye integración con red virtual (Microsoft, s.f.-a).
Azure Cosmos DB (serverless)	Base de datos NoSQL de pacientes, citas e historias clínicas	Esquema flexible para historias heterogéneas, latencia baja y cobro por unidad de solicitud (RU) consumida; resulta más barato que la capacidad aprovisionada en cargas intermitentes (Microsoft, s.f.-g).
Azure Blob Storage	Almacenamiento de resultados, imágenes y documentos	Niveles de acceso Hot, Cool, Cold y Archive con reglas automáticas de ciclo de vida, además de políticas de inmutabilidad para evidencias clínicas.
Azure Service Bus	Cola de eventos de citas y notificaciones	Desacopla la reserva del envío de mensajes, reintenta ante fallos y aísla los mensajes no procesables en una cola de mensajes fallidos (dead-letter).
Azure Communication Services	SMS, correo electrónico y videollamada	Un solo servicio para notificaciones y teleconsulta. El SMS hacia Perú se habilita con Messaging Connect, que amplía la cobertura a más de 190 países mediante socios (Microsoft, s.f.-e).

Nota. Elaboración propia. Servicios complementarios: Front Door, API Management, Static Web Apps, Key Vault, Azure Monitor, Defender for Cloud y Cost Management.

El SMS es un requisito crítico del caso: en muchas comunidades rurales los pacientes usan teléfonos básicos sin datos móviles. Por eso el canal de notificación se elige según el perfil del paciente: SMS para quienes no tienen smartphone y correo o notificación en la PWA para los demás, lo que además reduce costos.

2.4. Estrategia de escalabilidad y automatización

Escalabilidad horizontal y elástica. Cada componente escala de forma independiente y automática, sin intervención del equipo (Tabla 3). Como la arquitectura no guarda estado en el cómputo (las funciones no conservan sesión y todo el estado vive en Cosmos DB y Blob Storage), es posible agregar instancias en paralelo sin coordinación.

Tabla 3

Mecanismo de escalado por componente

Componente	Mecanismo de escalado	Comportamiento en una campaña
Front Door + Static Web Apps	Red perimetral (edge) global con caché	El contenido estático se sirve desde la caché y no consume cómputo.
API Management (Consumption)	Serverless, sin capacidad fija	Absorbe picos de solicitudes; los límites por usuario evitan abusos.
Azure Functions (Flex)	Escalado por concurrencia de solicitudes y por longitud de cola	Agrega instancias en segundos y vuelve a cero al terminar; para la jornada de campaña se pueden configurar instancias «siempre listas» y evitar arranques en frío.
Cosmos DB	Serverless (hasta 5000 RU/s por contenedor)	Si el consumo supera unos 100 millones de RU al mes, se migra a capacidad aprovisionada con escalado automático.

Componente	Mecanismo de escalado	Comportamiento en una campaña
Service Bus + fn-notificador	Nivelación de carga mediante cola	Los envíos masivos se encolan y se procesan al ritmo que admite el proveedor de SMS.

Nota. Elaboración propia con base en Microsoft (s.f.-a) y Microsoft (s.f.-g).

Infraestructura como código. Toda la infraestructura se declara con **Terraform** (proveedor *azurerm*), organizada en módulos (identidad, red, datos, cómputo, mensajería y monitoreo) y con estado remoto en Blob Storage. Así se pueden recrear entornos idénticos de desarrollo y producción, revisar los cambios antes de aplicarlos y recuperar la plataforma completa ante un desastre (Morris, 2020).

Integración y despliegue continuos (CI/CD). Se usa **Azure Pipelines** con dos pipelines (Anexo A): (a) en cada *pull request*: pruebas unitarias, análisis estático de seguridad (SAST), escaneo de dependencias y `terraform plan`; (b) al fusionar con la rama principal: despliegue en desarrollo, pruebas de humo, aprobación manual y despliegue en producción. La autenticación del pipeline ante Azure usa una conexión de servicio con federación OIDC, por lo que no se almacenan contraseñas ni secretos en el repositorio.

Alternativa evaluada: Kubernetes. Se consideró Azure Kubernetes Service (AKS). Ofrece portabilidad y control fino (Burns et al., 2022), pero exige mantener un clúster con nodos encendidos permanentemente y personal especializado, lo que contradice la restricción presupuestal. Si en el futuro se necesitaran contenedores, por ejemplo para un servicio de inteligencia artificial de apoyo al triaje, la opción intermedia sería Azure Container Apps, que también escala a cero.

2.5. Seguridad y cumplimiento

La historia clínica es un dato personal sensible. La seguridad se diseña con el enfoque de **confianza cero** (*Zero Trust*): verificar siempre la identidad, aplicar el mínimo privilegio y suponer que una brecha es posible. Los controles de la aplicación siguen el OWASP Top 10 (OWASP Foundation, 2021).

- **Identidad y control de acceso.** MFA obligatoria para médicos y opcional para pacientes (código por SMS o correo); control de acceso basado en roles (RBAC) en la aplicación, de modo que un médico solo ve a los pacientes asignados. Los servicios se comunican entre sí con identidades administradas (*Managed Identities*), sin contraseñas en el código.
- **Cifrado.** En tránsito, TLS 1.2 o superior en todos los extremos. En reposo, AES-256 del servicio en Cosmos DB y Blob Storage, con opción de llaves administradas por el cliente (CMK) en Key Vault. Además, el contenido clínico (diagnóstico, indicaciones, receta y signos vitales) se cifra en la aplicación con

AES-256-GCM antes de guardarse, con una llave que vive en Key Vault: quien acceda directamente a la base solo ve texto cifrado.

- **Red.** Cosmos DB y Storage solo aceptan tráfico mediante *Private Endpoints*; el WAF de Front Door aplica reglas de limitación de solicitudes y filtrado geográfico.
- **Monitoreo y auditoría.** Azure Monitor y Log Analytics registran cada acceso a una historia clínica (quién, cuándo y desde qué dirección IP); Microsoft Defender for Cloud evalúa la postura de seguridad y emite alertas de amenazas.
- **Integraciones.** Los socios externos usan claves de integración de las que solo se guarda el hash SHA-256, comparado en tiempo constante; la fachada FHIR aplica minimización de datos.
- **Gobierno.** Azure Policy asigna la iniciativa integrada *HIPAA/HITRUST*, que evalúa de forma continua los controles técnicos, y una política que restringe las regiones permitidas (Microsoft, s.f.-f).

La Tabla 4 relaciona los requisitos normativos aplicables con los controles de la arquitectura. HIPAA es una ley estadounidense y no rige en el Perú, pero se usa como referencia internacional de buenas prácticas, tal como sugiere el enunciado del caso.

Tabla 4

Correspondencia entre requisitos normativos y controles implementados

Norma	Requisito principal	Control en la arquitectura
Ley N.° 29733 y su Reglamento (D.S. 016-2024-JUS)	Consentimiento, medidas de seguridad y regulación del flujo transfronterizo de datos personales	Consentimiento informado en el registro, cifrado con CMK, Private Endpoints y evaluación del flujo transfronterizo (datos alojados en Chile y Brasil).
Ley N.° 30024 (modificada por la Ley N.° 31750)	Confidencialidad, accesibilidad e interoperabilidad de las historias clínicas electrónicas	RBAC por médico asignado, auditoría de accesos con IP, fachada FHIR R4 implementada y servidor FHIR en la fase 2 para interoperar con el RENHICE.
Ley N.° 30421, D. Leg. N. ° 1490 y su reglamento (D.S. N.° 005-2021-SA)	Telesalud como estrategia para ampliar la cobertura	Teleconsulta por video con opción de solo audio según la conexión medida.
HIPAA Security Rule (referencia)	Salvaguardas administrativas, físicas y técnicas	Iniciativa HIPAA/HITRUST en Azure Policy, registros de auditoría y backup continuo con restauración a un punto en el tiempo.

Nota. Elaboración propia a partir de Congreso de la República del Perú (2011, 2013, 2016, 2023), Presidencia de la República del Perú (2020), Ministerio de Salud del Perú (2021), Ministerio de Justicia y Derechos Humanos (2024) y U.S. Department of Health and Human Services (2022).

2.6. Optimización de costos

La Tabla 5 estima el costo mensual para un escenario base de 3000 pacientes activos, 1500 teleconsultas de 15 minutos, 4500 SMS y 100 GB de documentos. Los precios son referenciales, en dólares estadounidenses y sin impuestos; deben confirmarse en la Calculadora de precios de Azure para la región elegida.

Tabla 5

Estimación referencial de costo mensual en el escenario base

Servicio	Supuesto de uso	USD/mes (aprox.)
Entra External ID	3000 usuarios activos al mes (gratis hasta 50 000)	0
Static Web Apps (Standard)	Una aplicación con dominio propio	9
Front Door (Standard)	Tarifa base de USD 35 más tráfico reducido	40
API Management + Functions	Menos de 1 millón de llamadas; gran parte dentro de la cuota gratuita	5
Cosmos DB (serverless)	30 millones de RU y 10 GB a USD 0,25 cada uno	10
Blob Storage	100 GB repartidos entre los niveles Hot y Cool	3
Service Bus (Basic) + Key Vault	Mensajes y operaciones con llaves	2
ACS: videollamadas	1500 × 15 min × 2 participantes × USD 0,004	180
ACS: SMS (Messaging Connect)	4500 SMS a unos USD 0,04 cada uno (tarifa variable según el socio)	180
Monitor, Defender (CSPM básico) y Cost Management	Registros dentro de la cuota gratuita	5
Total estimado		**≈ 436**

Nota. Elaboración propia con tarifas publicadas por Microsoft (s.f.-a, s.f.-c, s.f.-e, s.f.-g). Los costos de comunicación son variables y crecen en proporción al número de consultas.

Más del 80 % del gasto corresponde a comunicaciones (video y SMS) y depende directamente de la actividad clínica; la infraestructura base cuesta alrededor de USD 75 al mes. En consecuencia, el costo marginal de una teleconsulta, sumando video y recordatorios, ronda los USD 0,25. Las medidas de control son las siguientes:

- **Pago por uso y escala a cero:** Functions, API Management y Cosmos DB en modo serverless no cobran cuando no hay actividad.
- **Almacenamiento escalonado:** una regla de ciclo de vida mueve los documentos a Cool a los 30 días, a Cold a los 90 y a Archive al año, lo que permite conservarlos durante el plazo legal a una fracción del costo.
- **Optimización de canales:** se envía SMS solo a quienes no tienen smartphone; la teleconsulta usa por defecto la resolución de video más baja aceptable y ofrece un modo de solo audio.
- **Monitoreo de consumo:** presupuestos en Cost Management con alertas al 50 %, 80 % y 100 %, y etiquetas (*tags*) por entorno y por campaña para atribuir el gasto a cada financiador.

2.7. Prototipo implementado y verificación

Para validar el diseño, el equipo construyó **SaludCerca**, un prototipo funcional de la arquitectura: una API en Python que se ejecuta en Azure Functions, una PWA en React, la infraestructura en Terraform y los pipelines de CI/CD en Azure Pipelines. También corre en

un computador con datos ficticios, sin costo, y se desplegó en Azure con el mismo código de Terraform. Como la suscripción Azure for Students no permite Front Door ni Static Web Apps y limita las regiones, la demostración usa la región Mexico Central y publica la PWA como sitio estático en Blob Storage; la arquitectura objetivo de la ONG no cambia. La Tabla 6 relaciona cada requisito del caso con el componente que lo resuelve y con la evidencia disponible en el prototipo.

Tabla 6

Trazabilidad entre requisitos, componentes y evidencia del prototipo

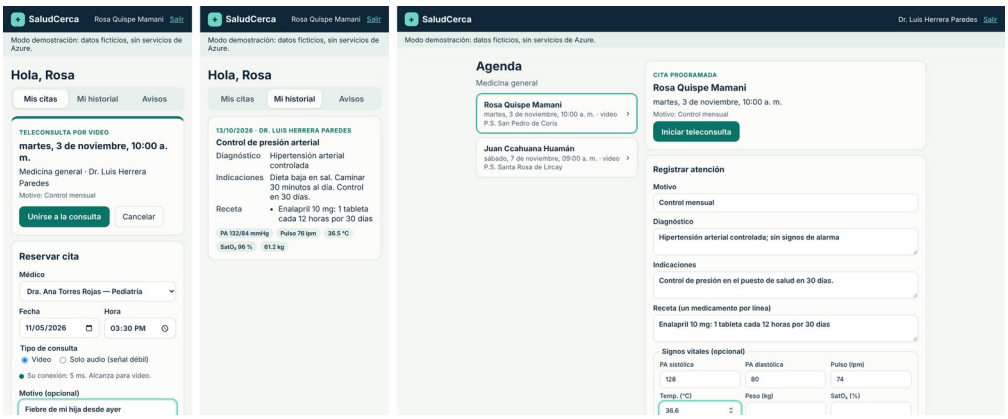
Requisito del caso	Componente	Evidencia en el prototipo
1. Autenticación segura	Entra External ID con MFA y roles; la API valida firma, emisor y audiencia del token	Pruebas automáticas: sin token responde 401; un rol sin permiso responde 403.
2. Historial confiable y accesible	Cosmos DB con backup continuo; contenido clínico cifrado con AES-256-GCM y llave en Key Vault	Prueba: el diagnóstico y la receta no aparecen en el registro guardado; el texto cifrado no se puede copiar a otro paciente.
3. Mensajería de citas	Service Bus y Communication Services; SMS solo para teléfonos básicos	Confirmación inmediata y recordatorio programado 24 h antes; el paciente con teléfono básico recibe SMS.
4. Escalado elástico	Functions Flex Consumption con tope de instancias, Cosmos DB serverless y cola	API sin estado; tope y memoria por instancia definidos como variables de Terraform.
5. Costos eficientes	Pago por uso, almacenamiento escalonado, presupuesto con alertas y cuota de registros	Recursos de Terraform: política de ciclo de vida, presupuesto 50/80/100 % y cuota diaria de 1 GB.
Interoperabilidad	Fachada FHIR R4 con clave de integración	Pruebas: Patient y Appointment responden sin datos clínicos; sin clave válida responde 401.

Nota. Elaboración propia.

Una función pensada para el contexto rural es la **medición real de la conexión**: al reservar y al entrar a la consulta, la app mide el tiempo de respuesta del servidor y, si es lento, preselecciona la consulta solo por audio (Figura 2). El médico registra la atención de forma estructurada, con receta y signos vitales, que el paciente ve descifrados en su historial. La teleconsulta usa una videollamada real de Azure Communication Services. Además, cada médico define sus bloques semanales de atención y el paciente reserva por especialidad entre los turnos libres (con al menos una hora de anticipación y hasta 30 días). Si no hay turnos, puede pedir que se le avise: cuando un médico de esa especialidad se registra o abre horario, el sistema le envía un único aviso por SMS, correo o la aplicación.

Figura 2

Prototipo SaludCerca (datos ficticios): reserva con medición de la conexión, historial del paciente y registro de la atención por el médico



La verificación combinó 64 casos de prueba automatizados de la API (autenticación, permisos por rol, reserva y doble reserva, notificaciones, cifrado, auditoría y FHIR) y un recorrido de punta a punta con Playwright en un navegador real por las pantallas de paciente, médico y coordinación. El flujo de integración continua repite en cada cambio esas pruebas, la validación de Terraform y el análisis de seguridad con CodeQL.

2.8. Gestión ágil del desarrollo

El prototipo se construyó con Scrum (Schwaber & Sutherland, 2020) en Azure Boards (Microsoft, s.f.-h), en tres sprints de una semana entre el 19 de octubre y el 8 de noviembre de 2026. El backlog se organizó en 6 épicas, 26 Issues y 64 Tasks; los criterios de aceptación de cada Issue se escribieron en Gherkin (Cucumber, s.f.) y luego se tradujeron a pruebas automatizadas. Los sprints 1 y 2 completaron los 34 puntos comprometidos cada uno, y durante el Sprint 3 el CI/CD se migró de GitHub Actions a Azure Pipelines para concentrar planificación, código y entrega en Azure DevOps.

Para respetar la extensión del documento, el detalle se presenta en los anexos: el Anexo A muestra la configuración del proyecto, el backlog, los tableros, los gráficos de burndown y velocidad, los pipelines y las pruebas por módulo; el Anexo B, los Issues y Tasks de cada sprint, y el Anexo C, los criterios de aceptación en Gherkin.

3. Conclusiones

La arquitectura PaaS/serverless propuesta en Azure responde a los cinco requisitos del caso. La autenticación segura se resuelve con Entra External ID y MFA; el historial clínico se guarda de forma confiable en Cosmos DB y Blob Storage, con cifrado y copias continuas; las notificaciones se envían por Service Bus y Communication Services, con SMS para teléfonos básicos; la escalabilidad es elástica y automática en todas las capas; y

el costo fijo es bajo, porque la mayor parte del gasto varía con el uso real. El prototipo SaludCerca demuestra que el diseño es implementable y deja pruebas automáticas como evidencia de cada requisito.

La gestión con Scrum en Azure Boards permitió entregar incrementos verificables en sprints de una semana, con trazabilidad entre el backlog, los criterios de aceptación en Gherkin y el código del repositorio, y una velocidad estable de 34 puntos por sprint.

El principal beneficio para la ONG es que puede operar una plataforma de nivel profesional con un equipo técnico pequeño y pagar en función de la actividad clínica. La infraestructura como código y el CI/CD con Azure Pipelines hacen que el sistema sea reproducible, auditable y fácil de recuperar ante desastres.

La implementación enfrenta, sin embargo, retos importantes: (a) la conectividad rural, que obliga a diseñar el modo sin conexión y el modo de solo audio desde el inicio; (b) el costo de las comunicaciones, que se vuelve el rubro dominante y debe negociarse con los socios de SMS; (c) el flujo transfronterizo de datos, porque no existe una región de Azure en el Perú, lo que exige cumplir el Reglamento de la Ley N.° 29733; (d) la interoperabilidad con el RENHICE, que requiere adoptar el estándar FHIR; y (e) la adopción por parte de médicos y pacientes, que demanda capacitación y una interfaz muy sencilla. Se recomienda empezar con un piloto en una región sanitaria, medir el costo por consulta y luego ampliar el alcance de forma gradual.

4. Referencias

Burns, B., Beda, J., Hightower, K., & Evenson, L. (2022). *Kubernetes: Up and running* (3.ª ed.). O'Reilly Media.

Congreso de la República del Perú. (2011). *Ley N.° 29733, Ley de Protección de Datos Personales*. Diario Oficial El Peruano.

Congreso de la República del Perú. (2013). *Ley N.° 30024, Ley que crea el Registro Nacional de Historias Clínicas Electrónicas*. Diario Oficial El Peruano.

Congreso de la República del Perú. (2016). *Ley N.° 30421, Ley Marco de Telesalud*. Diario Oficial El Peruano.

Congreso de la República del Perú. (2023). *Ley N.° 31750, Ley que modifica los artículos 2 y 4 de la Ley 30024, Ley que crea el Registro Nacional de Historias Clínicas Electrónicas*. Diario Oficial El Peruano. https://busquedas.elperuano.pe/dispositivo/NL/2180595-1

Cucumber. (s.f.). *Gherkin reference*. Recuperado el 3 de noviembre de 2026, de https://cucumber.io/docs/gherkin/reference/

Curioso, W. H., & Galán-Rodas, E. (2020). El rol de la telesalud en la lucha contra el COVID-19 y la evolución del marco normativo peruano. *Acta Médica Peruana, 37*(3), 366–375. https://doi.org/10.35663/amp.2020.373.1004

HL7 International. (2019). *FHIR release 4 (v4.0.1)*. https://hl7.org/fhir/R4/

Mell, P., & Grance, T. (2011). *The NIST definition of cloud computing* (NIST Special Publication 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

Microsoft. (s.f.-a). *Azure Functions pricing*. Recuperado el 2 de noviembre de 2026, de https://azure.microsoft.com/pricing/details/functions/

Microsoft. (s.f.-b). *Azure Well-Architected Framework*. Microsoft Learn. Recuperado el 2 de noviembre de 2026, de https://learn.microsoft.com/azure/well-architected/

Microsoft. (s.f.-c). *Compare pricing between Azure Front Door tiers*. Microsoft Learn. Recuperado el 2 de noviembre de 2026, de https://learn.microsoft.com/azure/frontdoor/understanding-pricing

Microsoft. (s.f.-d). *Overview of Microsoft Entra External ID in external tenants*. Microsoft Learn. Recuperado el 2 de noviembre de 2026, de https://learn.microsoft.com/entra/external-id/customers/overview-customers-ciam

Microsoft. (s.f.-e). *Pricing scenarios: Azure Communication Services*. Microsoft Learn. Recuperado el 2 de noviembre de 2026, de https://learn.microsoft.com/azure/communication-services/concepts/pricing

Microsoft. (s.f.-f). *Regulatory compliance details for HIPAA HITRUST*. Microsoft Learn. Recuperado el 2 de noviembre de 2026, de https://learn.microsoft.com/azure/governance/policy/samples/hipaa-hitrust

Microsoft. (s.f.-g). *Understand your Azure Cosmos DB bill*. Microsoft Learn. Recuperado el 2 de noviembre de 2026, de https://learn.microsoft.com/azure/cosmos-db/understand-your-bill

Microsoft. (s.f.-h). *What is Azure Boards?* Microsoft Learn. Recuperado el 3 de noviembre de 2026, de https://learn.microsoft.com/azure/devops/boards/get-started/what-is-azure-boards

Ministerio de Justicia y Derechos Humanos. (2024, 30 de noviembre). *Decreto Supremo N.° 016-2024-JUS, que aprueba el Reglamento de la Ley N.° 29733, Ley de Protección de Datos Personales*. https://www.gob.pe/institucion/anpd/normas-legales/6554453-16-2024-jus

Ministerio de Salud del Perú. (2021). *Decreto Supremo N.° 005-2021-SA, que aprueba el Reglamento de la Ley N.° 30421, Ley Marco de Telesalud, y del Decreto Legislativo N.° 1490*. Diario Oficial El Peruano. https://busquedas.elperuano.pe/dispositivo/NL/1922320-2

Morris, K. (2020). *Infrastructure as code: Dynamic systems for the cloud age* (2.ª ed.). O'Reilly Media.

OWASP Foundation. (2021). *OWASP Top 10:2021*. https://owasp.org/Top10/

Presidencia de la República del Perú. (2020). *Decreto Legislativo N.° 1490, que fortalece los alcances de la telesalud*. Diario Oficial El Peruano. https://busquedas.elperuano.pe/dispositivo/NL/1866212-2

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. https://scrumguides.org/scrum-guide.html

U.S. Department of Health and Human Services. (2022). *Summary of the HIPAA Security Rule*. https://www.hhs.gov/hipaa/for-professionals/security/laws-regulations/index.html

Anexo A. Gestión ágil del desarrollo en Azure Boards

El desarrollo del prototipo se gestionó con Scrum (Schwaber & Sutherland, 2020) en Azure Boards, el módulo de planificación de Azure DevOps (Microsoft, s.f.-h). Como el curso duró tres semanas, el trabajo se organizó en tres sprints de una semana; así cada iteración entregó un incremento verificable. El proyecto DesarrolloNube, de la organización curiosity00, contiene el tablero de trabajo y el repositorio Git del prototipo. Esto permite relacionar cada elemento de trabajo con el código, la infraestructura y los pipelines que lo implementan. La Tabla 7 resume la configuración.

Tabla 7

Configuración del proyecto en Azure DevOps

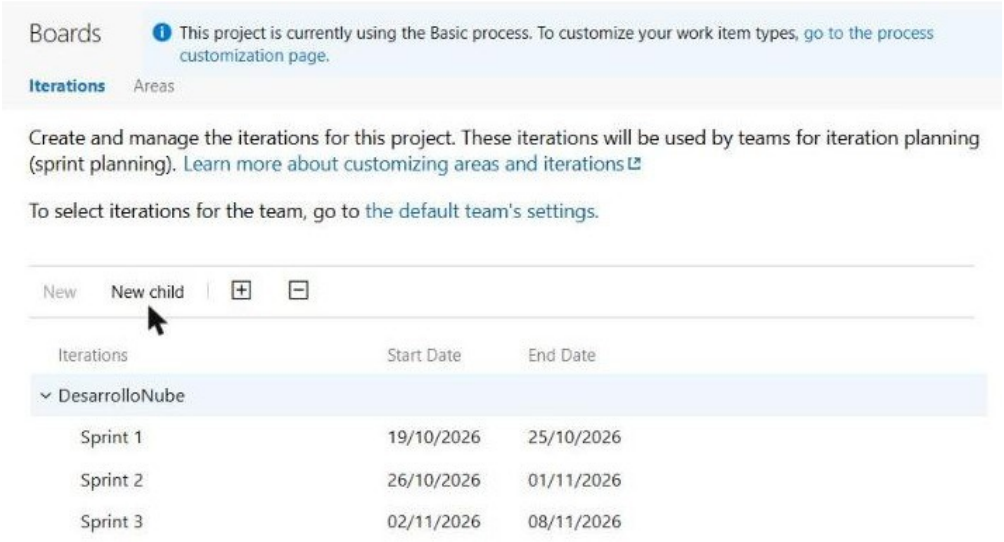
Elemento	Configuración
Organización / proyecto	curiosity00 / DesarrolloNube (Boards y Repos)
Proceso de trabajo	Basic: Epic → Issue → Task, con estados To Do, Doing y Done
Estimación	Effort (puntos relativos) en Issues; horas de trabajo restante en Tasks
Iteraciones	Sprint 1 (19/10–25/10), Sprint 2 (26/10–01/11) y Sprint 3 (02/11–08/11) de 2026
Capacidad	5 horas diarias por integrante, 50 horas por sprint para el equipo
Definición de Listo	Historia de usuario, criterios en Gherkin, Effort estimado y épica padre
Definición de Hecho	Código integrado en main, pruebas de CI aprobadas, escenarios Gherkin cumplidos y Tasks en Done

Nota. Elaboración propia.

El primer paso fue crear las tres iteraciones del proyecto con sus fechas de inicio y fin y asignarlas al equipo (Figura 3). Luego se registró la capacidad de cada integrante en cada sprint, que sirve de referencia para no comprometer más horas de las disponibles (Figura 4).

Figura 3

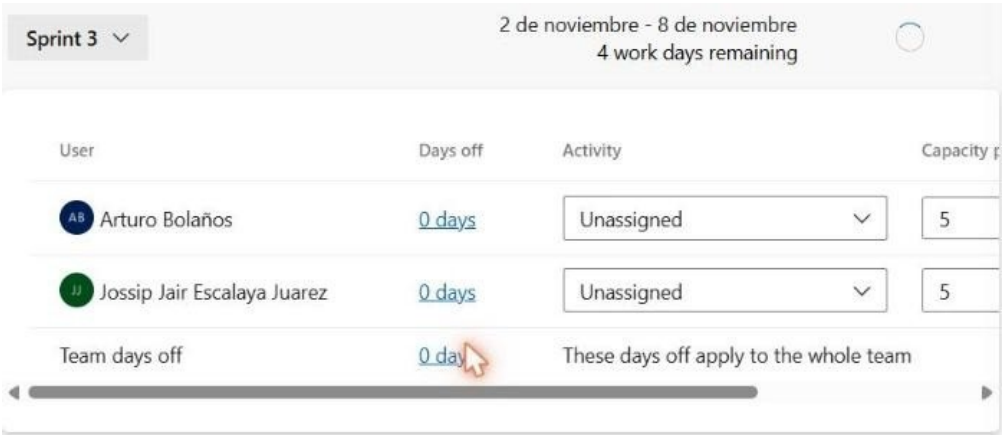
Iteraciones del proyecto configuradas en Azure DevOps



Nota. Captura de Project settings > Boards > Project configuration.

Figura 4

Capacidad del equipo registrada para el Sprint 3



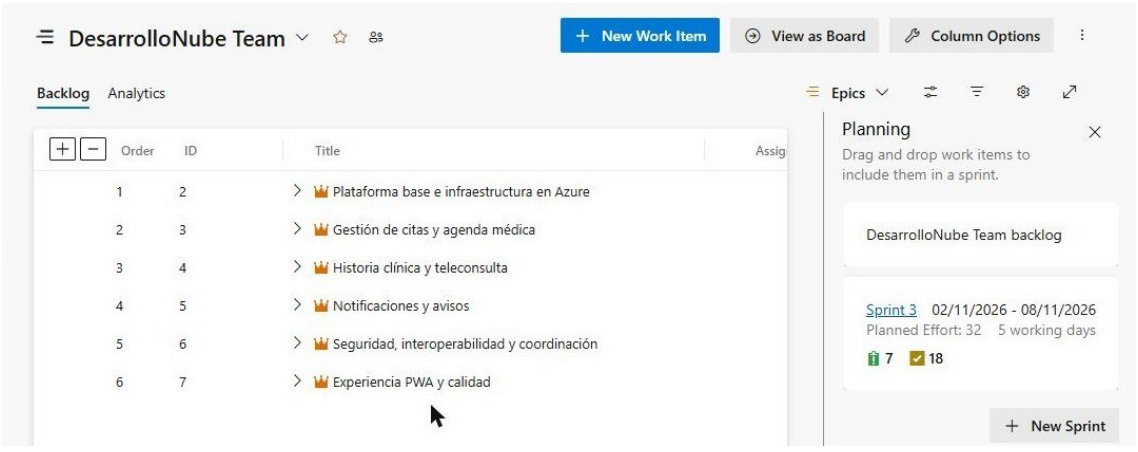
Nota. Captura de la pestaña Capacity del Sprint 3; la capacidad se expresa en horas por día.

A.1. Product Backlog

El backlog se construyó a partir de la arquitectura de las secciones anteriores y se organizó en seis épicas (Figura 5): plataforma base e infraestructura, gestión de citas y agenda, historia clínica y teleconsulta, notificaciones y avisos, seguridad e interoperabilidad, y experiencia PWA y calidad. Cada épica se dividió en Issues con valor verificable y cada Issue en Tasks técnicas estimadas en horas (Figura 6). En total se registraron 6 épicas, 26 Issues y 64 Tasks; la Tabla 8 presenta los Issues y su estado al 3 de noviembre de 2026.

Figura 5

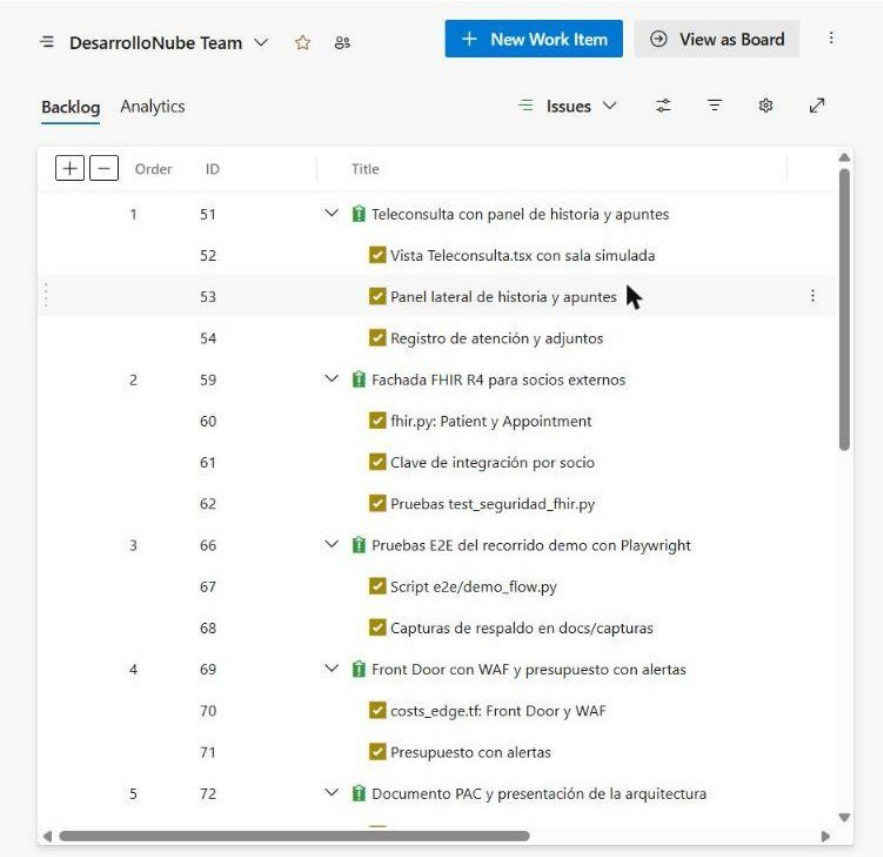
Backlog de épicas del proyecto



Nota. Captura del backlog del equipo DesarrolloNube Team en el nivel Epics.

Figura 6

Backlog de Issues con sus Tasks



Nota. Vista jerárquica del backlog: cada Issue agrupa las Tasks técnicas que lo implementan.

Tabla 8

Issues del Product Backlog

ID	Issue	Épica	Sprint	Responsable	Effort	Estado
9	Estructura del repositorio y entorno local con Docker Compose	E1	1	A. Bolaños	3	Done
12	Infraestructura base con Terraform (Functions, Cosmos DB, Storage)	E1	1	A. Bolaños	8	Done
16	Pipeline de CI con GitHub Actions	E1	1	A. Bolaños	5	Done
19	Autenticación con Entra External ID y roles	E5	1	J. Escalaya	8	Done
23	PWA base en React y TypeScript con navegación por rol	E6	1	J. Escalaya	5	Done
26	Modelo de datos y repositorio en Cosmos DB particionado por paciente	E2	1	A. Bolaños	5	Done
30	Horario de atención del médico	E2	2	J. Escalaya	5	Done
33	Reserva de cita: especialidad, médico, día y turno	E2	2	J. Escalaya	8	Done
37	Historia clínica cifrada con AES-256-GCM y Key Vault	E3	2	A. Bolaños	8	Done
41	Confirmación y recordatorio 24 h con Service Bus	E4	2	A. Bolaños	5	Done

ID	Issue	Épica	Sprint	Responsable	Effort	Estado
45	Modo sin conexión con service worker	E6	2	J. Escalaya	3	Done
48	Pipeline de despliegue con OIDC y aprobación manual	E1	2	A. Bolaños	5	Done
51	Teleconsulta con panel de historia y apuntes	E3	3	J. Escalaya	8	Doing
55	Lista de espera y aviso de turnos disponibles	E4	3	A. Bolaños	5	Done
59	Fachada FHIR R4 para socios externos	E5	3	A. Bolaños	5	Doing
63	Medición de conexión y recomendación de solo audio	E2	3	J. Escalaya	3	Done
66	Pruebas E2E del recorrido demo con Playwright	E6	3	J. Escalaya	3	To Do
69	Front Door con WAF y presupuesto con alertas	E5	3	A. Bolaños	3	To Do
72	Documento PAC y presentación de la arquitectura	E6	3	J. Escalaya	5	Doing
94	Migración de CI/CD de GitHub Actions a Azure Pipelines	E1	3	J. Escalaya	5	Doing
76	Videollamada real con Azure Communication Services	E3	Backlog	—	8	To Do
77	Private Endpoints e integración con red virtual	E5	Backlog	—	8	To Do
78	Llaves administradas por el cliente (CMK) en Cosmos DB y Storage	E5	Backlog	—	5	To Do
79	Rotación de la llave de cifrado de notas clínicas	E3	Backlog	—	5	To Do
80	API Management delante de la Function App	E5	Backlog	—	5	To Do
81	Remitente SMS hacia Perú con Messaging Connect	E4	Backlog	—	3	To Do

Nota. E1: Plataforma base e infraestructura; E2: Gestión de citas y agenda; E3: Historia clínica y teleconsulta; E4: Notificaciones y avisos; E5: Seguridad, interoperabilidad y coordinación; E6: Experiencia PWA y calidad. El detalle de Tasks por sprint se presenta en el Anexo B.

A.2. Criterios de aceptación en Gherkin

Los criterios de aceptación de cada Issue se escribieron en Gherkin, el lenguaje de especificación por ejemplos de Cucumber (Cucumber, s.f.), con sus palabras clave en español: Característica, Escenario, Dado, Cuando, Entonces e Y. Cada característica parte de una historia de usuario (Como… quiero… para…) y sus escenarios fijan el contexto, la acción y el resultado esperado, de modo que se pueden traducir directamente a pruebas automatizadas con pytest o Playwright. Los escenarios se registraron en la descripción de cada Issue (Figura 7); a continuación se muestra el del Issue 33 y el conjunto completo está en el Anexo C.

Issue 33

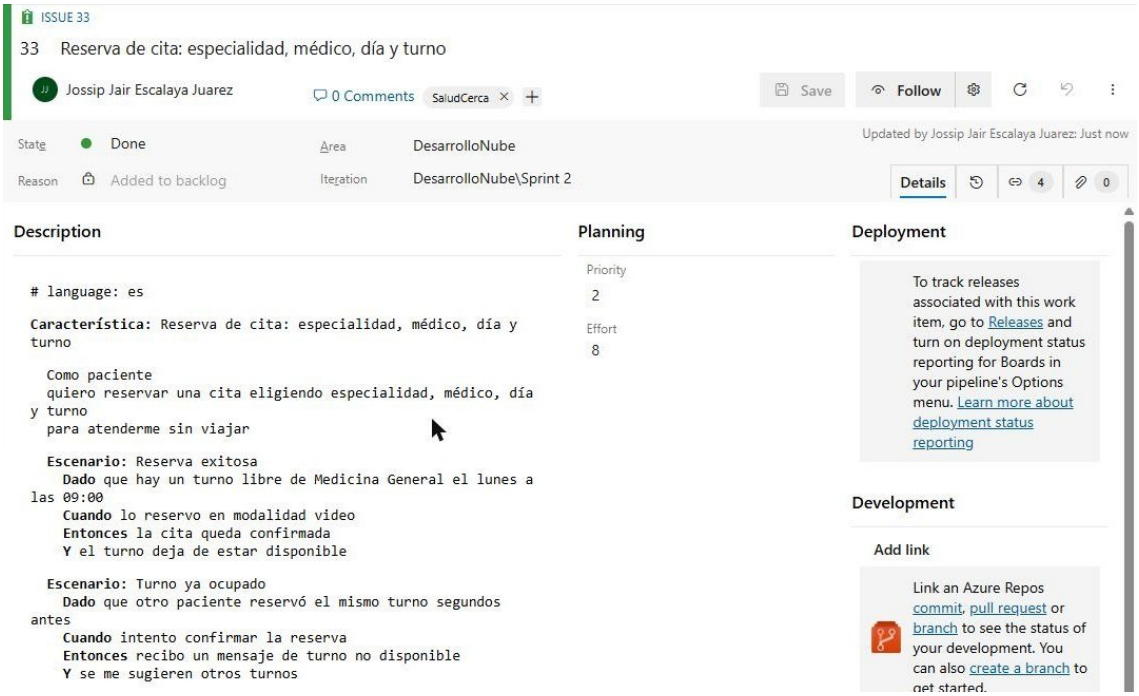
Característica: Reserva de cita: especialidad, médico, día y turno
Como paciente
quiero reservar una cita eligiendo especialidad, médico, día y turno
para atenderme sin viajar

Escenario: Reserva exitosa
Dado que hay un turno libre de Medicina General el lunes a las 09:00
Cuando lo reservo en modalidad video
Entonces la cita queda confirmada
Y el turno deja de estar disponible

Escenario: Turno ya ocupado
Dado que otro paciente reservó el mismo turno segundos antes
Cuando intento confirmar la reserva
Entonces recibo un mensaje de turno no disponible
Y se me sugieren otros turnos

Figura 7

Issue 33 con sus criterios de aceptación en Gherkin



Nota. Vista de detalle del Issue en Azure Boards con su estado, sprint, Effort y épica padre.

A.3. Ejecución de los sprints

Cada sprint comenzó con una planificación en la que se seleccionaron Issues del backlog según su prioridad y dependencias, y se descompusieron en Tasks con horas estimadas. Durante el sprint, el avance se registró en el tablero, moviendo cada elemento entre To Do, Doing y Done y actualizando el trabajo restante; al cierre se revisó el incremento y se acordaron mejoras para el siguiente sprint. La Tabla 9 resume los objetivos y resultados.

Tabla 9

Objetivos y resultados de los sprints

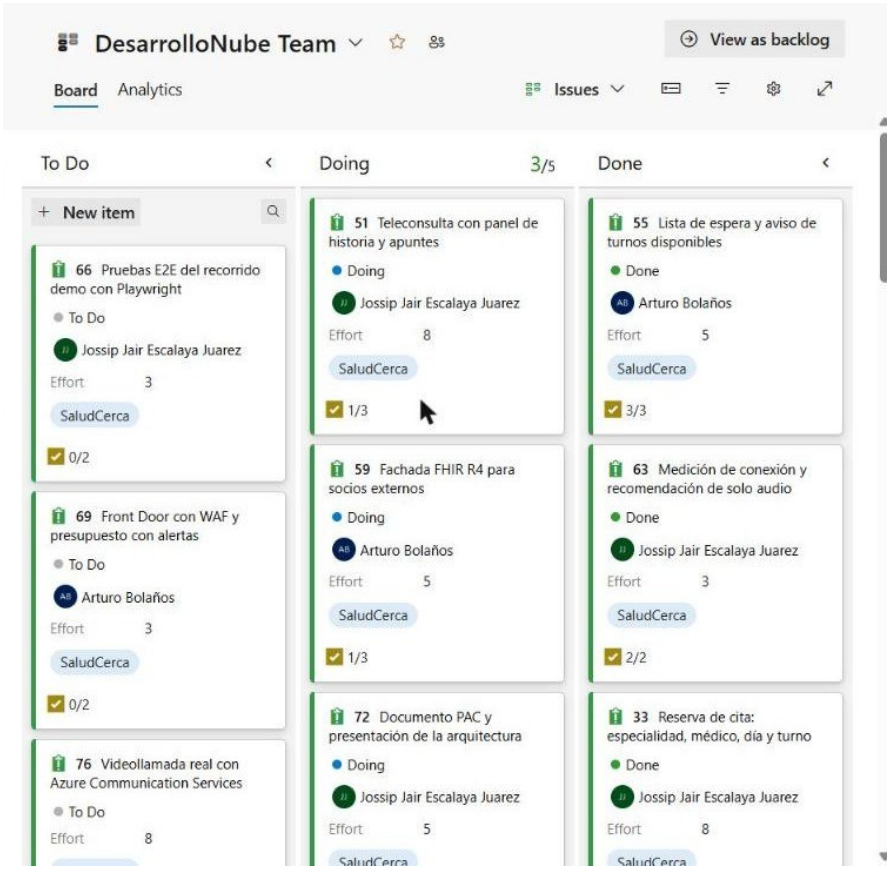
Sprint	Objetivo	Issues	Effort compr. / compl.	Tasks (h)	Resultado
Sprint 1 (19/10–25/10)	Base técnica: repositorio, entorno local, Terraform, CI, autenticación con roles y modelo de datos	6	34 / 34	15 (49 h)	Completado
Sprint 2 (26/10–01/11)	Flujo principal: horario del médico, reserva, historia clínica cifrada, notificaciones, modo sin conexión y despliegue continuo	6	34 / 34	15 (55 h)	Completado
Sprint 3 (02/11–08/11)	Atención remota e interoperabilidad: teleconsulta, lista de espera, FHIR R4, Front Door, pruebas E2E, documentación y migración a Azure Pipelines	8	37 / 8	22 (67 h)	En curso: 2 Issues terminados y 4 en progreso

Nota. Effort comprometido y completado en puntos. Datos de Azure Boards al 3 de noviembre de 2026.

El tablero Kanban muestra el estado de cada Issue y el avance de sus Tasks (Figura 8). Los gráficos de burndown de los sprints 1 y 2 (Figuras 9 y 10) muestran que el trabajo restante bajó más rápido que la tendencia ideal y llegó a cero antes del cierre; en ambos sprints se completaron los 34 puntos comprometidos. La Figura 11 muestra el Taskboard del Sprint 3, en curso, y la Figura 12 la velocidad del equipo.

Figura 8

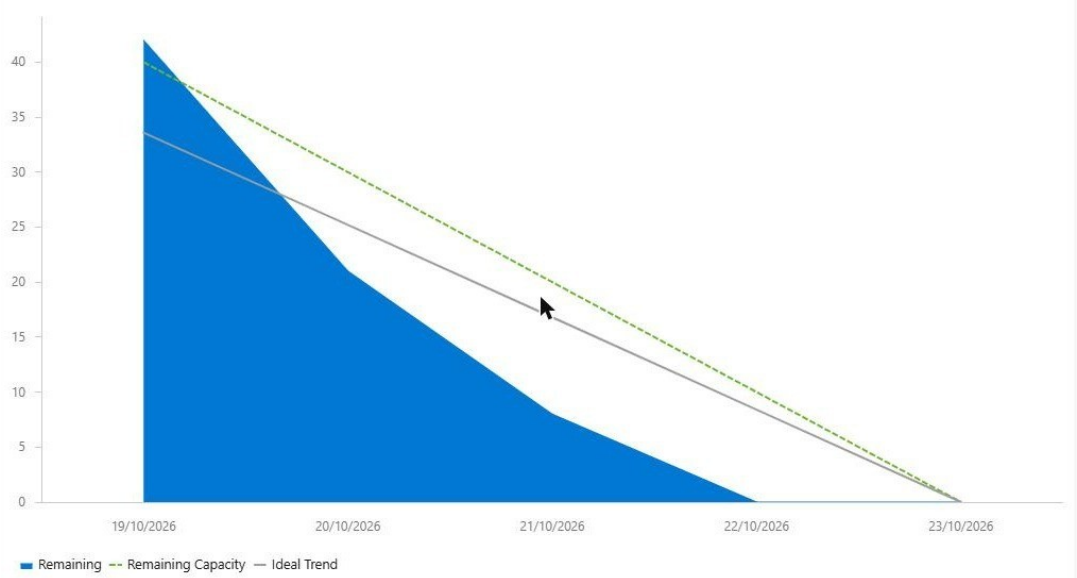
Tablero Kanban de Issues



Nota. El indicador inferior de cada tarjeta muestra las Tasks terminadas sobre el total.

Figura 9

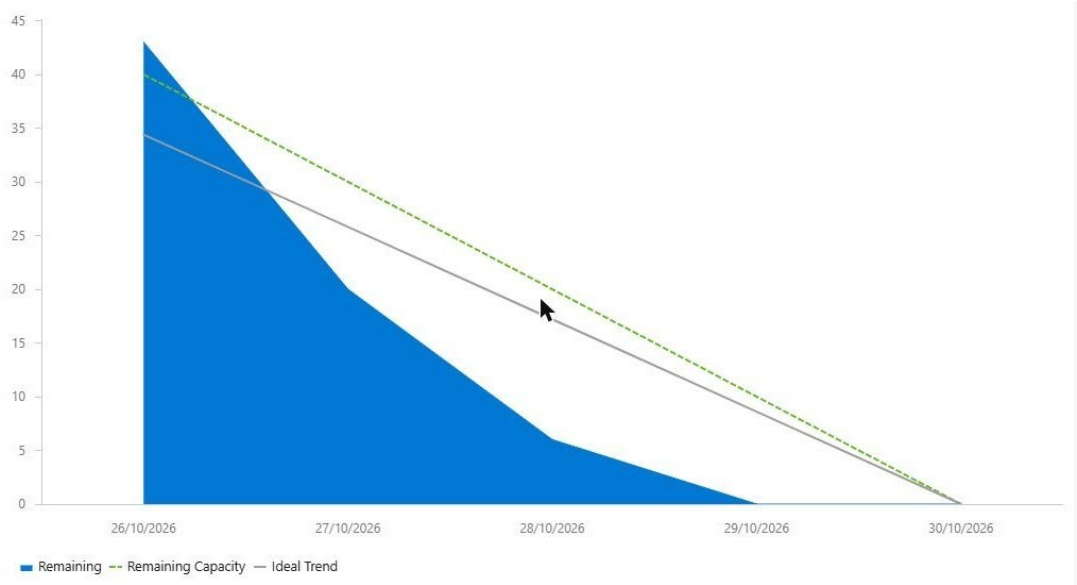
Burndown del Sprint 1



Nota. Reporte Burndown Trend sobre el trabajo restante en horas. La línea gris es la tendencia ideal y la verde, la capacidad restante.

Figura 10

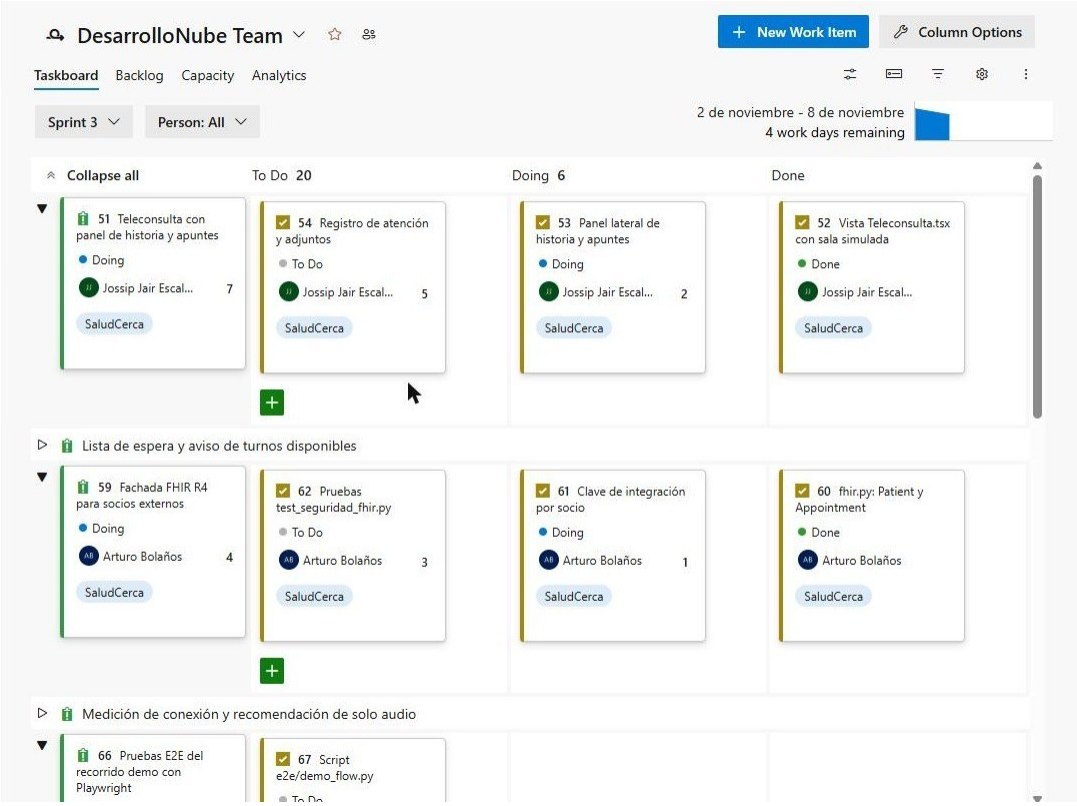
Burndown del Sprint 2



Nota. Reporte Burndown Trend del Sprint 2.

Figura 11

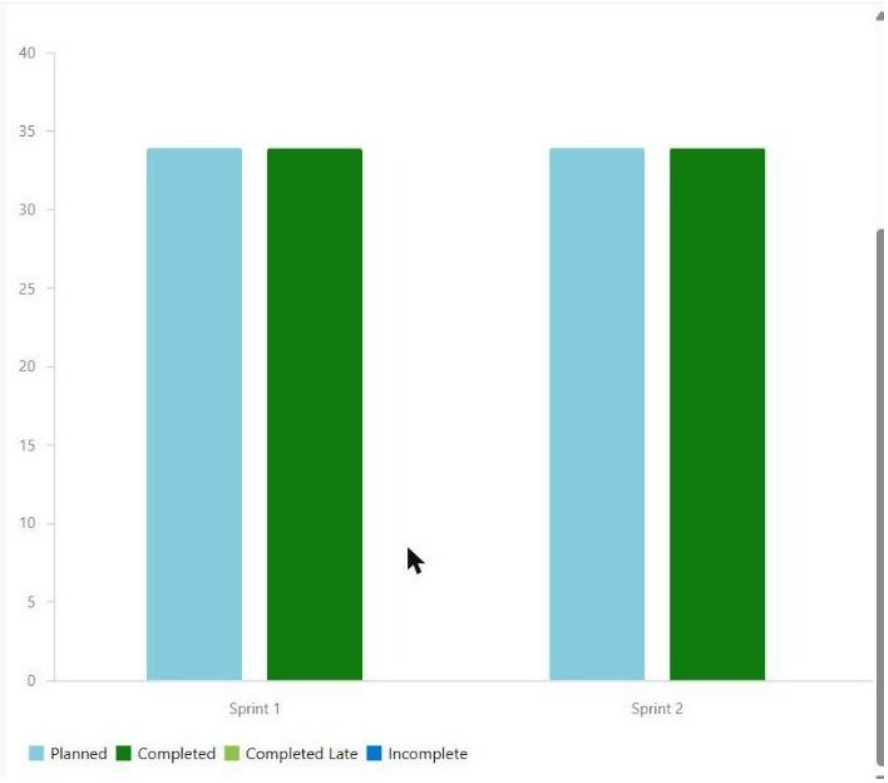
Taskboard del Sprint 3



Nota. Cada fila corresponde a un Issue y sus Tasks se distribuyen en To Do, Doing y Done; el número de cada tarjeta es el trabajo restante en horas.

Figura 12

Velocidad del equipo por sprint



Nota. Reporte Velocity con la suma de Effort planificado y completado en los sprints cerrados.

La velocidad fue de 34 puntos en cada sprint cerrado, por lo que el compromiso inicial del Sprint 3 (32 puntos) quedó dentro de lo observado; durante el sprint se incorporó además la migración del CI/CD a Azure Pipelines (5 puntos). Como lecciones, el equipo identificó que escribir los criterios en Gherkin antes de programar alineó el código con las pruebas, que conviene planificar primero las dependencias de infraestructura y que la carga debe ajustarse a la capacidad: en el Sprint 2 se planificaron 55 horas para 50 disponibles. Después de ese corte, el 4 de noviembre se completaron la videollamada real con ACS (Issue 76) y el recorrido E2E con Playwright (Issue 66), y la plataforma se desplegó en Azure con Terraform. Los cinco Issues restantes (Private Endpoints, llaves administradas por el cliente, rotación de llaves, API Management y SMS hacia Perú) quedaron estimados en el backlog para iteraciones futuras.

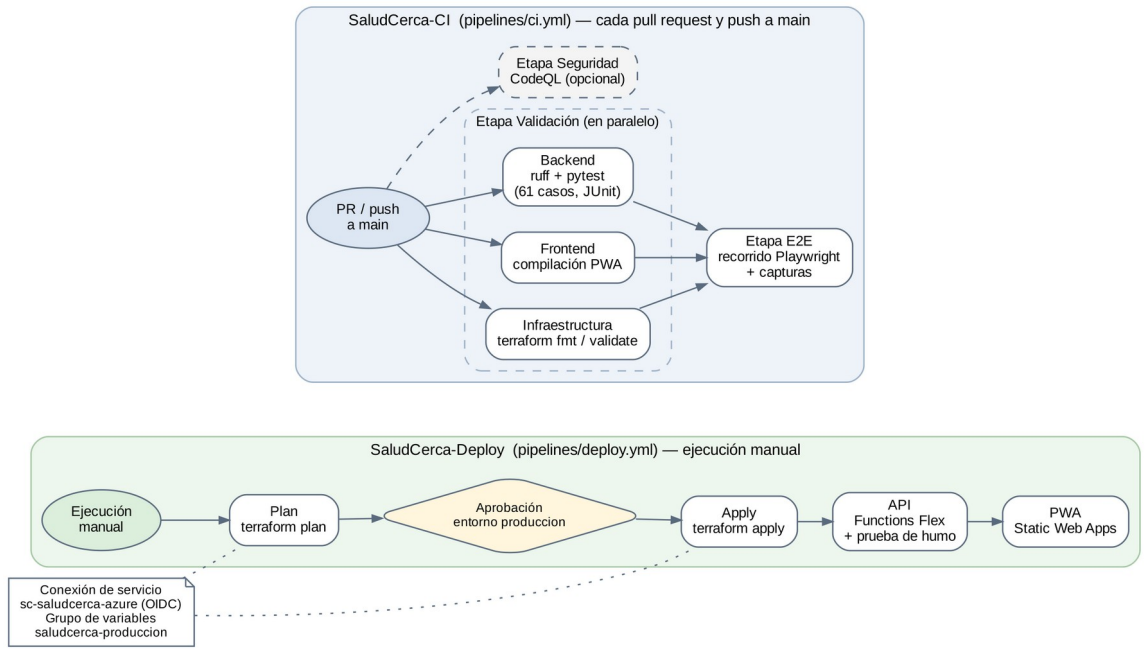
A.4. Integración y despliegue continuos con Azure Pipelines

Para concentrar la planificación, el código y la entrega en una sola plataforma, los flujos de CI/CD definidos inicialmente en GitHub Actions (Issue 16) se migraron a Azure Pipelines dentro del mismo proyecto de Azure DevOps. La migración conservó las mismas acciones y se registró como Issue 94 del Sprint 3, vinculado al pull request #1. Se definieron dos pipelines en YAML, versionados en la carpeta pipelines/ del repositorio: SaludCerca-CI,

que valida cada cambio, y SaludCerca-Deploy, que despliega la plataforma en Azure (Figura 13). La Tabla 10 muestra la correspondencia entre ambos.

Figura 13

Flujo de los pipelines de integración y despliegue continuos



Nota. Elaboración propia a partir de los archivos pipelines/ci.yml y pipelines/deploy.yml. La línea punteada indica los recursos de Azure DevOps que usa el despliegue.

Tabla 10

Correspondencia entre los flujos de GitHub Actions y los pipelines de Azure Pipelines

Acción	GitHub Actions	Azure Pipelines
Disparador de CI	on: pull_request y push a main	trigger y pr sobre main
Pruebas del backend	ruff y pytest	ruff y pytest con resultados JUnit publicados en la pestaña Tests
Compilación de la PWA	npm run build	NodeTool + npm run build
Validación de infraestructura	terraform fmt y validate	terraform fmt y validate
Recorrido de punta a punta	Playwright + artefacto de capturas	Playwright + PublishPipelineArtifact
Análisis estático de seguridad	CodeQL	Tareas de Advanced Security (CodeQL), activables por variable
Autenticación ante Azure	Federación OIDC	Conexión de servicio con identidad federada (OIDC)
Variables y secretos	Variables y secrets del repositorio	Grupo de variables saludcerca-produccion
Aprobación manual	Entorno produccion protegido	Entorno produccion con verificación de aprobación
Despliegue de la API	Functions Flex con compilación remota	az functionapp deployment source config-zip --build-remote
Despliegue de la PWA	static-web-apps-deploy	AzureCLI@2 con scripts/publicar-web.sh (sitio estático en Blob Storage)

Acción	GitHub Actions	Azure Pipelines
Ejecución sin solapamientos	concurrency	lockBehavior: sequential

Nota. Elaboración propia.

El siguiente fragmento de pipelines/ci.yml muestra la estructura de etapas del pipeline de integración continua:

```
trigger:
  branches:
    include: [main]
pr:
  branches:
    include: [main]
stages:
  - stage: Validacion       # Backend, Frontend e Infraestructura en paralelo
  - stage: E2E              # recorrido de la demo con Playwright
    dependsOn: Validacion
  - stage: Seguridad        # CodeQL con Advanced Security
    condition: eq(variables['habilitarCodeQL'], true)
```

Las pruebas automatizadas del backend suman 64 casos, distribuidos en seis módulos (Tabla 11). El pipeline de despliegue ya está definido, pero su primera ejecución requiere configurar tres recursos en Azure DevOps: la conexión de servicio sc-saludcerca-azure con identidad federada, el grupo de variables saludcerca-produccion (estado de Terraform, entorno, identificadores de Entra External ID y el hash de las claves de integración, este último como secreto) y el entorno produccion con aprobación obligatoria. Hasta entonces se ejecuta solo de forma manual, sin disparador automático.

Tabla 11

Casos de prueba automatizados del backend por módulo

Módulo de prueba	Alcance	Casos
test_api.py	API: autenticación, permisos por rol y flujos principales	17
test_pruebas_locales.py	Modo de pruebas local con inicio de sesión OIDC (incluye una prueba parametrizada con 5 casos)	12
test_avisos.py	Lista de espera y avisos de disponibilidad	11
test_seguridad_fhir.py	Seguridad y fachada FHIR R4	10
test_agenda.py	Agenda del médico y turnos libres	9
test_perfil.py	Perfil del usuario (Mis datos)	5
Total	59 funciones de prueba	64

Nota. Elaboración propia a partir del directorio backend/tests del repositorio. Además, el recorrido de punta a punta (e2e/demo_flow.py) valida en un navegador real los flujos de paciente, médico y coordinación.

Anexo B. Issues y Tasks por sprint

Tabla 12

Issues y Tasks del Sprint 1

ID	Elemento de trabajo	Responsable	Estimación	Estado
9	Issue: Estructura del repositorio y entorno local con Docker Compose	A. Bolaños	3 pts	Done
10	Task: Crear estructura backend, frontend e infra	A. Bolaños	2 h	Done
11	Task: Dockerfiles y docker-compose.yml	A. Bolaños	3 h	Done
12	Issue: Infraestructura base con Terraform (Functions, Cosmos DB, Storage)	A. Bolaños	8 pts	Done
13	Task: Function App Flex Consumption y Storage	A. Bolaños	5 h	Done
14	Task: Cosmos DB serverless y contenedores	A. Bolaños	4 h	Done
15	Task: Backend remoto del estado de Terraform	A. Bolaños	2 h	Done
16	Issue: Pipeline de CI con GitHub Actions	A. Bolaños	5 pts	Done
17	Task: Workflow ci.yml con pytest	A. Bolaños	3 h	Done
18	Task: Build del frontend en CI	A. Bolaños	2 h	Done
19	Issue: Autenticación con Entra External ID y roles	J. Escalaya	8 pts	Done
20	Task: Tenant externo, registro de API y App Roles	J. Escalaya	4 h	Done
21	Task: MSAL en el frontend (auth.ts)	J. Escalaya	4 h	Done
22	Task: Validación del JWT en el backend (auth.py)	J. Escalaya	4 h	Done
23	Issue: PWA base en React y TypeScript con navegación por rol	J. Escalaya	5 pts	Done
24	Task: Scaffold React + Vite + TypeScript	J. Escalaya	2 h	Done
25	Task: Login y enrutamiento por rol	J. Escalaya	4 h	Done
26	Issue: Modelo de datos y repositorio en Cosmos DB particionado por paciente	A. Bolaños	5 pts	Done
27	Task: Modelos Pydantic (models.py)	A. Bolaños	3 h	Done
28	Task: CosmosRepository y repositorio en memoria	A. Bolaños	5 h	Done
29	Task: Datos ficticios de demo (seed.py)	A. Bolaños	2 h	Done

Nota. Las Tasks aparecen debajo del Issue al que pertenecen.

Tabla 13

Issues y Tasks del Sprint 2

ID	Elemento de trabajo	Responsable	Estimación	Estado
30	Issue: Horario de atención del médico	J. Escalaya	5 pts	Done
31	Task: API de agenda y disponibilidad (agenda.py)	J. Escalaya	5 h	Done
32	Task: Vista Horario.tsx	J. Escalaya	4 h	Done
33	Issue: Reserva de cita: especialidad, médico, día y turno	J. Escalaya	8 pts	Done
34	Task: Endpoint de reserva y validación de turno libre	J. Escalaya	5 h	Done
35	Task: Vista Reserva.tsx	J. Escalaya	5 h	Done
36	Task: Pruebas test_agenda.py	J. Escalaya	3 h	Done
37	Issue: Historia clínica cifrada con AES-256-GCM y Key Vault	A. Bolaños	8 pts	Done
38	Task: Módulo cifrado.py	A. Bolaños	4 h	Done

ID	Elemento de trabajo	Responsable	Estimación	Estado
39	Task: Key Vault y Managed Identity (security.tf)	A. Bolaños	4 h	Done
40	Task: Vista Historial.tsx	A. Bolaños	4 h	Done
41	Issue: Confirmación y recordatorio 24 h con Service Bus	A. Bolaños	5 pts	Done
42	Task: messaging.py con mensajes programados	A. Bolaños	4 h	Done
43	Task: Disparador fn-notificador	A. Bolaños	3 h	Done
44	Task: AcsNotifier para SMS y correo	A. Bolaños	3 h	Done
45	Issue: Modo sin conexión con service worker	J. Escalaya	3 pts	Done
46	Task: Service worker sw.js	J. Escalaya	3 h	Done
47	Task: manifest.webmanifest y caché	J. Escalaya	2 h	Done
48	Issue: Pipeline de despliegue con OIDC y aprobación manual	A. Bolaños	5 pts	Done
49	Task: deploy.yml con plan, aprobación y apply	A. Bolaños	4 h	Done
50	Task: Despliegue de API y PWA con prueba de humo	A. Bolaños	2 h	Done

Nota. Las Tasks aparecen debajo del Issue al que pertenecen.

Tabla 14

Issues y Tasks del Sprint 3

ID	Elemento de trabajo	Responsable	Estimación	Estado
51	Issue: Teleconsulta con panel de historia y apuntes	J. Escalaya	8 pts	Doing
52	Task: Vista Teleconsulta.tsx con sala simulada	J. Escalaya	5 h	Done
53	Task: Panel lateral de historia y apuntes	J. Escalaya	4 h	Doing
54	Task: Registro de atención y adjuntos	J. Escalaya	5 h	To Do
55	Issue: Lista de espera y aviso de turnos disponibles	A. Bolaños	5 pts	Done
56	Task: avisos.py y evento de disponibilidad en la cola	A. Bolaños	4 h	Done
57	Task: Botón Avisarme en la reserva	A. Bolaños	2 h	Done
58	Task: Pruebas test_avisos.py	A. Bolaños	2 h	Done
59	Issue: Fachada FHIR R4 para socios externos	A. Bolaños	5 pts	Doing
60	Task: fhir.py: Patient y Appointment	A. Bolaños	4 h	Done
61	Task: Clave de integración por socio	A. Bolaños	2 h	Doing
62	Task: Pruebas test_seguridad_fhir.py	A. Bolaños	3 h	To Do
63	Issue: Medición de conexión y recomendación de solo audio	J. Escalaya	3 pts	Done
64	Task: conexion.ts: medición de latencia	J. Escalaya	2 h	Done
65	Task: Sugerencia de solo audio en reserva y consulta	J. Escalaya	2 h	Done
66	Issue: Pruebas E2E del recorrido demo con Playwright	J. Escalaya	3 pts	To Do
67	Task: Script e2e/demo_flow.py	J. Escalaya	3 h	To Do
68	Task: Capturas de respaldo en docs/capturas	J. Escalaya	1 h	To Do
69	Issue: Front Door con WAF y presupuesto con alertas	A. Bolaños	3 pts	To Do
70	Task: costs_edge.tf: Front Door y WAF	A. Bolaños	3 h	To Do
71	Task: Presupuesto con alertas	A. Bolaños	2 h	To Do
72	Issue: Documento PAC y presentación de la arquitectura	J. Escalaya	5 pts	Doing
73	Task: Redactar el PAC de arquitectura	J. Escalaya	6 h	Doing
74	Task: Diagrama de arquitectura	J. Escalaya	3 h	Done

ID	Elemento de trabajo	Responsable	Estimación	Estado
75	Task: Diapositivas de la exposición	J. Escalaya	3 h	To Do
94	Issue: Migración de CI/CD de GitHub Actions a Azure Pipelines	J. Escalaya	5 pts	Doing
96	Task: Convertir ci.yml a Azure Pipelines	J. Escalaya	3 h	Done
97	Task: Convertir deploy.yml a Azure Pipelines	J. Escalaya	4 h	Done
98	Task: Crear las definiciones SaludCerca-CI y SaludCerca-Deploy	J. Escalaya	1 h	Done
95	Task: Configurar conexión de servicio, grupo de variables y entorno produccion	J. Escalaya	3 h	To Do

Nota. Las Tasks aparecen debajo del Issue al que pertenecen.

Tabla 15

Issues y Tasks del backlog para iteraciones futuras

ID	Elemento de trabajo	Responsable	Estimación	Estado
76	Issue: Videollamada real con Azure Communication Services	—	8 pts	To Do
82	Task: Integrar SDK @azure/communication-calling	—	8 h	To Do
83	Task: Permisos de cámara y micrófono y reconexión	—	4 h	To Do
77	Issue: Private Endpoints e integración con red virtual	—	8 pts	To Do
84	Task: Red virtual y subredes en Terraform	—	5 h	To Do
85	Task: Private Endpoints para Cosmos DB, Storage y Key Vault	—	5 h	To Do
78	Issue: Llaves administradas por el cliente (CMK) en Cosmos DB y Storage	—	5 pts	To Do
86	Task: Llaves en Key Vault e identidad para cifrado	—	3 h	To Do
87	Task: CMK en Cosmos DB y Storage	—	4 h	To Do
79	Issue: Rotación de la llave de cifrado de notas clínicas	—	5 pts	To Do
88	Task: Identificador de llave en el formato v2	—	4 h	To Do
89	Task: Proceso por lotes de recifrado	—	5 h	To Do
80	Issue: API Management delante de la Function App	—	5 pts	To Do
90	Task: API Management en Terraform	—	4 h	To Do
91	Task: Políticas de límite de tasa y claves	—	3 h	To Do
81	Issue: Remitente SMS hacia Perú con Messaging Connect	—	3 pts	To Do
92	Task: Habilitar remitente con Messaging Connect	—	2 h	To Do
93	Task: Configurar acs_sms_from y probar envío	—	2 h	To Do

Nota. Las Tasks aparecen debajo del Issue al que pertenecen.

Anexo C. Criterios de aceptación en Gherkin

Sprint 1

Issue 9

Característica: Estructura del repositorio y entorno local con Docker Compose
Como desarrollador del equipo
quiero levantar toda la solución con un solo comando
para trabajar sin depender de Azure

Escenario: Levantar el entorno local
Dado que clono el repositorio y tengo Docker instalado
Cuando ejecuto docker compose up --build
Entonces se inician los contenedores web y api
Y GET /api/health responde 200

Issue 12

Característica: Infraestructura base con Terraform (Functions, Cosmos DB, Storage)
Como ingeniero de plataforma
quiero definir la infraestructura como código
para recrear el entorno de forma repetible

Escenario: Plan de infraestructura válido
Dado que configuro las variables del entorno en terraform.tfvars
Cuando ejecuto terraform plan
Entonces el plan se genera sin errores
Y incluye Function App, Cosmos DB serverless y Storage con etiquetas estándar

Escenario: Estado remoto compartido
Dado que existe la cuenta de Storage para el estado
Cuando dos miembros ejecutan terraform init
Entonces ambos usan el mismo estado remoto con bloqueo

Issue 16

Característica: Pipeline de CI con GitHub Actions
Como equipo de desarrollo
quiero validar cada cambio automáticamente
para detectar errores antes de integrar

Escenario: Validación en cada push
Dado que hago push a una rama del repositorio
Cuando se ejecuta el workflow de CI
Entonces corren las pruebas del backend con pytest
Y se compila el frontend
Y el pipeline falla si alguna prueba falla

Issue 19

Característica: Autenticación con Entra External ID y roles
Como usuario de SaludCerca
quiero iniciar sesión de forma segura
para acceder solo a las funciones de mi rol

Escenario: Inicio de sesión con MFA
Dado que soy un paciente registrado
Cuando ingreso mi correo y el código de un solo uso
Y confirmo el segundo factor por SMS
Entonces accedo a la pantalla de inicio de paciente

Escenario: Token inválido rechazado
Dado que una solicitud llega con un JWT vencido o de otro emisor
Cuando la API valida el token
Entonces responde 401 y no procesa la solicitud

Issue 23

Característica: PWA base en React y TypeScript con navegación por rol
Como usuario autenticado
quiero ver una pantalla de inicio según mi rol
para encontrar rápido lo que necesito

Escenario: Inicio según rol
Dado que inicio sesión con el rol Medico
Cuando la aplicación carga
Entonces veo la pantalla MedicoHome
Y no veo opciones de Coordinación

Issue 26

Característica: Modelo de datos y repositorio en Cosmos DB particionado por paciente
Como desarrollador backend
quiero persistir citas e historias por paciente
para consultar sus datos con una sola partición

Escenario: Guardar con clave de partición
Dado que se registra una cita para un paciente
Cuando el repositorio la guarda en Cosmos DB
Entonces el documento usa el id del paciente como clave de partición

Escenario: Pruebas sin Azure
Dado que ejecuto las pruebas en local
Cuando la API se inicia sin conexión a Cosmos DB
Entonces usa el repositorio en memoria con los datos de demo

Sprint 2

Issue 30

Característica: Horario de atención del médico
Como médico
quiero definir mi horario de atención
para que los pacientes solo reserven cuando atiendo

Escenario: Publicar horario
Dado que soy médico autenticado
Cuando defino lunes a viernes de 08:00 a 12:00 con turnos de 20 minutos
Entonces se generan turnos libres solo dentro de esas franjas

Escenario: Día bloqueado
Dado que bloqueo el 15 de noviembre
Cuando un paciente consulta disponibilidad para ese día
Entonces no se muestran turnos

Issue 33

Característica: Reserva de cita: especialidad, médico, día y turno
Como paciente
quiero reservar una cita eligiendo especialidad, médico, día y turno
para atenderme sin viajar

Escenario: Reserva exitosa
Dado que hay un turno libre de Medicina General el lunes a las 09:00
Cuando lo reservo en modalidad video
Entonces la cita queda confirmada
Y el turno deja de estar disponible

Escenario: Turno ya ocupado
Dado que otro paciente reservó el mismo turno segundos antes
Cuando intento confirmar la reserva
Entonces recibo un mensaje de turno no disponible
Y se me sugieren otros turnos

Issue 37

Característica: Historia clínica cifrada con AES-256-GCM y Key Vault
Como paciente
quiero que mi información clínica esté protegida
para que solo la vea el personal autorizado

Escenario: Contenido cifrado en reposo
Dado que un médico registra un diagnóstico
Cuando se guarda en Cosmos DB
Entonces el contenido clínico se almacena cifrado con AES-256-GCM
Y la llave se obtiene de Key Vault con identidad administrada

Escenario: Acceso restringido
Dado que soy un médico que no atiende al paciente
Cuando intento abrir su historia clínica
Entonces recibo 403
Y el intento queda en el log de auditoría con mi IP

Issue 41

Característica: Confirmación y recordatorio 24 h con Service Bus
Como paciente
quiero recibir confirmación y recordatorio de mi cita
para no olvidar la teleconsulta

Escenario: Confirmación inmediata
Dado que reservo una cita
Cuando la reserva se confirma
Entonces se encola un evento en Service Bus
Y recibo la confirmación por mi canal preferido

Escenario: Recordatorio programado
Dado que mi cita es el viernes a las 10:00
Cuando llega el jueves a las 10:00
Entonces recibo un recordatorio por app, correo o SMS

Issue 45

Característica: Modo sin conexión con service worker
Como paciente en zona rural
quiero usar la app con conectividad intermitente
para consultar mis citas aunque pierda señal

Escenario: Abrir sin conexión
Dado que abrí la app al menos una vez con conexión
Cuando la abro sin red
Entonces se carga desde caché
Y veo mis próximas citas guardadas

Issue 48

Característica: Pipeline de despliegue con OIDC y aprobación manual
Como equipo de desarrollo
quiero desplegar a Azure de forma controlada
para evitar cambios no revisados en producción

Escenario: Despliegue con aprobación
Dado que se integra un cambio en main
Cuando se ejecuta el pipeline de despliegue
Entonces terraform plan espera la aprobación del entorno produccion
Y tras aprobar se despliegan la API y la PWA

Escenario: Prueba de humo
Dado que la API se desplegó
Cuando el pipeline llama a /api/health
Entonces el despliegue falla si no responde 200

Sprint 3

Issue 51

Característica: Teleconsulta con panel de historia y apuntes
Como médico
quiero consultar la historia y tomar apuntes durante la llamada
para atender sin cambiar de pantalla

Escenario: Panel durante la consulta
Dado que inicio la teleconsulta con mi paciente
Cuando abro el panel lateral
Entonces veo su historia clínica
Y puedo escribir apuntes no visibles para el paciente

Escenario: Registro de la atención
Dado que terminé la consulta
Cuando registro diagnóstico, indicaciones, receta y signos vitales
Entonces la atención queda en la historia del paciente

Issue 55

Característica: Lista de espera y aviso de turnos disponibles
Como paciente
quiero que me avisen cuando haya turnos
para no tener que revisar la app todos los días

Escenario: Aviso de disponibilidad
Dado que no hay turnos de Cardiología y pedí Avisarme cuando haya turnos
Cuando Coordinación registra un cardiólogo con horario
Entonces recibo un único aviso por mi canal

Escenario: Sin avisos duplicados
Dado que ya recibí el aviso
Cuando se publican más turnos de la misma especialidad
Entonces no recibo un segundo aviso

Issue 59

Característica: Fachada FHIR R4 para socios externos
Como socio externo (hospital o DIRESA)
quiero consultar pacientes y citas en FHIR R4
para integrar SaludCerca con mis sistemas

Escenario: Consulta con clave válida
Dado que tengo una clave de integración vigente
Cuando consulto /api/fhir/Patient/{id}
Entonces recibo un recurso Patient FHIR R4
Y no incluye contenido clínico

Escenario: Clave inválida
Dado que uso una clave inexistente
Cuando consulto /api/fhir/Appointment
Entonces recibo 401

Issue 63

Característica: Medición de conexión y recomendación de solo audio
Como paciente con señal débil
quiero que la app me sugiera solo audio
para no perder la consulta por cortes de video

Escenario: Señal débil
Dado que la latencia medida al servidor es alta
Cuando entro a reservar o a la consulta
Entonces la app me recomienda la modalidad solo audio

Issue 66

Característica: Pruebas E2E del recorrido demo con Playwright
Como equipo
quiero automatizar el recorrido de la demo
para asegurar que funcione en la exposición

Escenario: Recorrido completo
Dado que la aplicación corre en local
Cuando ejecuto e2e/demo_flow.py
Entonces se completa el flujo paciente, médico y coordinación sin errores
Y se guardan capturas en docs/capturas

Issue 69

Característica: Front Door con WAF y presupuesto con alertas
Como responsable de la ONG
quiero controlar costos y proteger el acceso público
para operar el piloto de forma sostenible

Escenario: Front Door opcional
Dado que habilitar_front_door es true
Cuando aplico Terraform
Entonces la PWA y la API quedan detrás de Front Door con WAF

Escenario: Alerta de presupuesto
Dado que el gasto mensual supera el 80 % del presupuesto
Cuando Azure evalúa el consumo
Entonces se envía una alerta al correo de la coordinación

Issue 72

Característica: Documento PAC y presentación de la arquitectura
Como equipo del curso
quiero documentar la arquitectura de la solución
para sustentarla en la exposición

Escenario: Entrega del PAC
Dado que la solución está implementada
Cuando se entrega el documento
Entonces incluye diagrama de arquitectura y justificación de cada servicio de Azure
Y las diapositivas resumen la propuesta

Issue 94

Característica: Migración de CI/CD de GitHub Actions a Azure Pipelines
Como equipo de desarrollo
quiero ejecutar la integración y el despliegue continuos en Azure Pipelines
para tener planificación, código y entrega en el mismo proyecto de Azure DevOps

Escenario: Validación de un pull request
Dado que abro un pull request hacia main
Cuando se ejecuta SaludCerca-CI
Entonces corren lint, pruebas, compilación de la PWA, validación de Terraform y el recorrido E2E
Y los resultados de pytest se publican en la pestaña Tests

Escenario: Despliegue con aprobación
Dado que existen la conexión de servicio, el grupo de variables y el entorno produccion
Cuando ejecuto SaludCerca-Deploy
Entonces el apply espera la aprobación del entorno produccion
Y tras aprobar se despliegan la API y la PWA

Backlog para iteraciones futuras

Issue 76 (completado el 4 de noviembre, después del corte)

Característica: Videollamada real con Azure Communication Services
Como médico y paciente
quiero tener una videollamada real
para atender sin la sala simulada

Escenario: Llamada real
Dado que el backend emite el token de ACS
Cuando ambos ingresan a la teleconsulta
Entonces se establece una videollamada con audio y video

Issue 77

Característica: Private Endpoints e integración con red virtual
Como responsable de seguridad
quiero que los datos no estén expuestos a internet
para reducir la superficie de ataque

Escenario: Acceso privado
Dado que los Private Endpoints están configurados
Cuando se intenta acceder a Cosmos DB desde internet
Entonces el acceso es denegado
Y la Function App accede por la red virtual

Issue 78

Característica: Llaves administradas por el cliente (CMK) en Cosmos DB y Storage
Como responsable de seguridad
quiero cifrar en reposo con llaves propias
para cumplir requisitos de protección de datos de salud

Escenario: Cifrado con CMK
Dado que existe una llave en Key Vault
Cuando se configura Cosmos DB y Storage
Entonces ambos cifran los datos con esa llave

Issue 79

Característica: Rotación de la llave de cifrado de notas clínicas
Como responsable de seguridad
quiero rotar la llave de cifrado
para limitar el impacto de una llave comprometida

Escenario: Rotación sin pérdida
Dado que existen notas cifradas con la llave v1
Cuando se ejecuta el recifrado por lotes con la llave v2
Entonces todas las notas se leen correctamente con la nueva llave

Issue 80

Característica: API Management delante de la Function App
Como socio externo
quiero consumir la API con límites claros
para tener un acceso estable y controlado

Escenario: Límite de tasa
Dado que mi suscripción permite 100 solicitudes por minuto
Cuando envío la solicitud 101
Entonces recibo 429

Issue 81

Característica: Remitente SMS hacia Perú con Messaging Connect
Como paciente con teléfono básico
quiero recibir SMS reales en mi número peruano
para enterarme de mis citas sin smartphone

Escenario: SMS real
Dado que mi canal preferido es SMS con número +51
Cuando se confirma mi cita
Entonces recibo el SMS de confirmación